

Deploying Trim

It's live (Vercel free tier, deployed 2026-07-13)

Project ref is not in this file. This repository is **public**, so the live Supabase project ref is kept in `DEPLOY.local.md` (gitignored) instead of here. Replace `YOUR-PROJECT-REF` below with the value from that file, or read it off the Supabase dashboard URL.

What	URL
The app (share this)	<code>https://trim-budget.vercel.app</code>
API	<code>https://trim-api-jade.vercel.app (/api/health to check)</code>
Vercel projects	<code>trim-budget</code> (client) + <code>trim-api</code> (server), account <code>alexander-quang-phan</code>

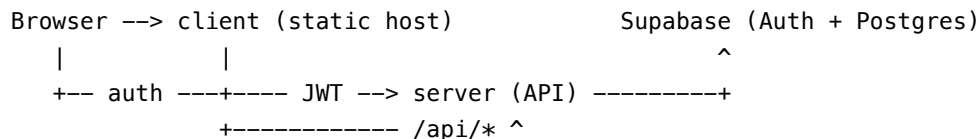
Both run on Vercel's free Hobby plan — \$0/mo. The server runs as a serverless function (`server/api/index.js` wraps the same Express app; `vercel.json` routes everything to it). Two free-tier caveats:

- **Rate limiting is per-function-instance** on serverless, so the 20-asks/hour cap is approximate rather than exact. Fine at friends-and-family scale.
- **Supabase free tier still pauses after ~1 week idle** (see §1). Daily use keeps it awake.

To redeploy after code changes: `cd server && vercel deploy --prod` and/or `cd client && vercel deploy --prod`. Env vars live in the Vercel dashboard → project → Settings → Environment Variables.

Remaining manual steps (2 min, Supabase dashboard): 1. Authentication → URL Configuration → Site URL → `https://trim-budget.vercel.app`. 2. Authentication → Passwords → enable leaked password protection. 3. Before sharing widely: Authentication → Sign In / Providers → Email → turn on “Confirm email”.

The rest of this guide is the original **Railway** path — a paid (~\$5/mo) alternative with a long-running server (exact rate limits, no serverless caveats). Use it if you outgrow the free tier.



0. What you need

- The GitHub repo for this project (push it first if it only lives locally).
- Your Supabase project (`Trim_Budgeting_App`, ref `YOUR-PROJECT-REF`) — already set up with migrations 001–009 applied.
- A [Railway](#) account (Hobby plan, ~\$5/mo, includes both services).
- Your Anthropic API key (powers Ask Trim + the AI quick-add parser).

1. Supabase — one-time checks

The database already exists. Before going live:

1. **Keep it awake.** Free-tier Supabase projects pause after ~1 week without traffic (this already bit us once — the DNS literally disappears). Either upgrade the project to Pro (\$10/mo), or accept that you must “Restore” it from the dashboard whenever it pauses.
2. **Auth settings** (Dashboard → Authentication):
 - *Sign In / Providers* → *Email*: leave email+password enabled.
 - *Rate limits*: defaults are fine.
 - *Passwords*: enable **leaked password protection** (flagged by the security advisor; one toggle).
 - *URL Configuration* → *Site URL*: set to your deployed client URL once you know it (step 3), e.g. `https://trim-client-production.up.railway.app`.
3. **Migrations on a fresh project** (only if you ever rebuild from zero): run `server/migrations/001...009_*.sql` in order in the SQL Editor.
4. Collect the four values you’ll need (Dashboard → Project Settings → API):
 - Project URL → `SUPABASE_URL` / `VITE_SUPABASE_URL`
 - anon / publishable key → `VITE_SUPABASE_ANON_KEY`
 - service_role key → `SUPABASE_SERVICE_ROLE_KEY` (server only, never the client)
 - Settings → API → JWT Secret → `SUPABASE_JWT_SECRET` (server only)

2. Railway — API service

1. Railway dashboard → **New Project** → **Deploy from GitHub repo** → pick this repo.
2. In the new service’s **Settings**:
 - **Root Directory**: `server`
 - Build/start are picked up from `server/railway.json` (start `npm start`, health check `/api/health`). No changes needed.

3. **Variables** tab — add:

Variable	Value
<code>NODE_ENV</code>	<code>production</code>
<code>CLIENT_URL</code>	placeholder for now — you’ll paste the real client URL in step 4
<code>SUPABASE_URL</code>	<code>https://YOUR-PROJECT-REF.supabase.co</code>
<code>SUPABASE_SERVICE_ROLE_KEY</code>	from step 1.4
<code>SUPABASE_JWT_SECRET</code>	from step 1.4
<code>ANTHROPIC_API_KEY</code>	your key — server service only
<code>ALLOWED_EMAILS</code>	every permitted account email, comma-separated — the server will not start without it

(Don’t set `PORT` — Railway injects it.)

4. **Settings** → **Networking** → **Generate Domain**. Note the URL, e.g. `https://trim-api-production.up.railway.app`.

5. Check it's alive: open `<api-url>/api/health` → `{"status":"ok",...}`.

3. Railway — client service

1. Same Railway project → + **New** → **GitHub Repo** → same repo again.
2. Service **Settings**:
 - **Root Directory**: `client`
 - Build/start come from `client/railway.json` (build `npm run build`, start `serve -s dist`).
3. **Variables** tab — add (these are baked in at build time):

Variable	Value
VITE_API_URL	the API URL from step 2.4, no trailing slash
VITE_SUPABASE_URL	<code>https://YOUR-PROJECT-REF.supabase.co</code>
VITE_SUPABASE_ANON_KEY	the anon/publishable key

4. **Settings** → **Networking** → **Generate Domain**. This is your app's address — the one you share with friends and family.

4. Point them at each other

1. Back on the **API service** → **Variables**: set `CLIENT_URL` to the client's domain from step 3.4 — exact origin, `https://`, **no trailing slash**. (CORS and the CSP `connect-src` both key off this; a mismatch = blank dashboard with console errors.)
2. Supabase → Authentication → URL Configuration → **Site URL** = same client URL.
3. Redeploy both services (Railway → service → Deployments → → Redeploy) so env changes take effect. The client must rebuild for `VITE_*` changes.

5. Smoke test (2 minutes)

1. Open the client URL on your phone and your laptop.
2. Sign up with a real email → you land on the Dashboard with 12 seeded categories and a friendly empty state.
3. Log a transaction with the + button (3 taps) → “+10 XP” toast.
4. Open **Ask Trim** (sparkle button, bottom-left) → ask “what did I spend this week?” → streamed answer.
5. Settings → toggle Simple mode on/off; set a monthly limit.

If anything fails, check the API service logs first (Railway → service → Logs): CORS errors mean `CLIENT_URL` doesn't match; 401s mean a wrong `SUPABASE_JWT_SECRET`.

6. Costs

Thing	Cost
Railway Hobby (both services)	~\$5/mo, includes usage credit
Supabase Free tier	\$0, pauses when idle · Pro \$10/mo never pauses

Thing	Cost
Anthropic API (Ask Trim + parser, Haiku 4.5)	pennies — a chat turn \$0.002; rate-limited to 20/user/hour

7. Sharing with friends & family (and beyond)

- Anyone with the URL can sign up — every user gets their own isolated data (RLS + per-user scoping on every query).
- Before sharing widely, consider in Supabase Auth: **enable email confirmation** (Authentication → Sign In / Providers → Email → Confirm email) so strangers can't squat on someone's address.
- If it takes off and you want to sell it: put the client behind a custom domain (Railway → client service → Settings → Custom Domain), upgrade Supabase to Pro, and revisit the **100 req / 15 min** global rate limit in `server/index.js` (it's per-IP; fine for dozens of users, tight for hundreds).

Env var reference

Variable	Where	Purpose
PORT	server (Railway injects)	HTTP port
NODE_ENV	server	production
CLIENT_URL	server	exact client origin for CORS/CSP
SUPABASE_URL	server + client (VITE_)	Supabase project URL
SUPABASE_SERVICE_ROLE_KEY	server only	data access (bypasses RLS)
SUPABASE_JWT_SECRET	server only	verify user JWTs
ANTHROPIC_API_KEY	server only	Ask Trim chat + NL quick-add parser
ASK_MODEL	server, optional	override Ask Trim model (default <code>claude-haiku-4-5</code>)
VITE_API_URL	client build	API base URL ("" in dev = Vite proxy)
VITE_SUPABASE_ANON_KEY	client build	Supabase Auth sign-in only